Stochastic Processes

Markov Decision Processes and Dynamic Programming

Jie Gong

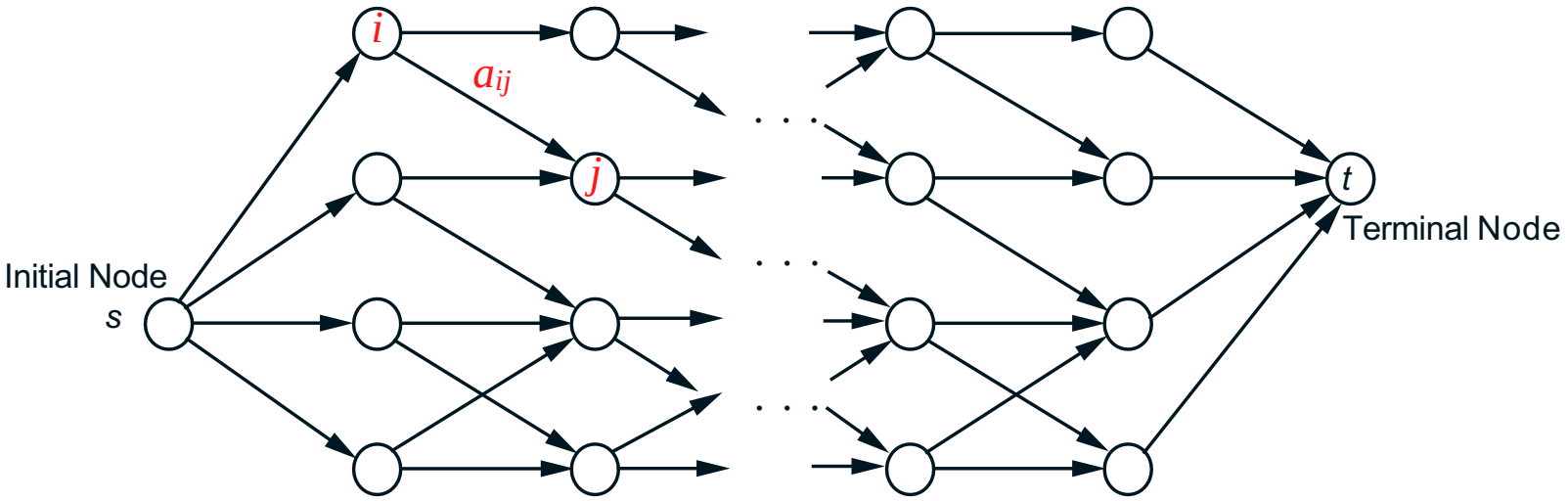
Autumn 2022

Outline

1. Shortest Path Problem
2. Markov Decision Process
3. Remarks

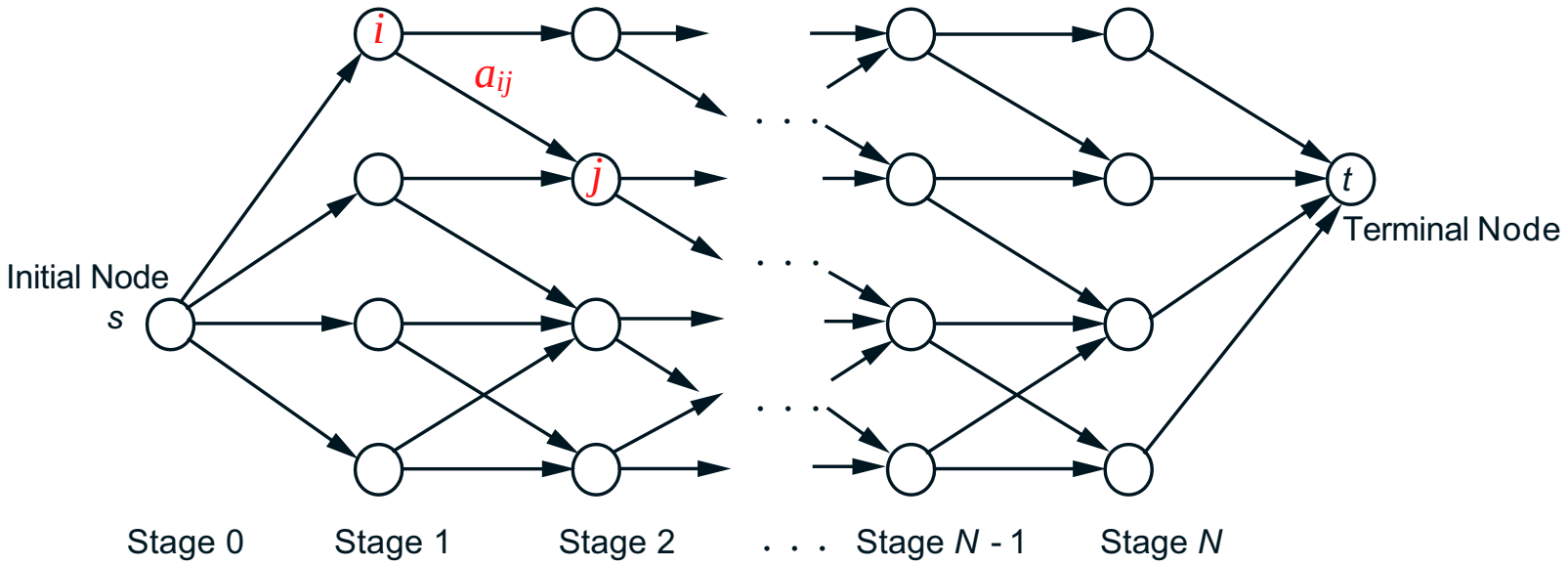
Shortest Path Problem

- A set of nodes between s and t
- A path between nodes i and j with length a_{ij}
- If no path, $a_{ij} = \infty$
- Objective: Finding the shortest path from s to t



Problem Formulation

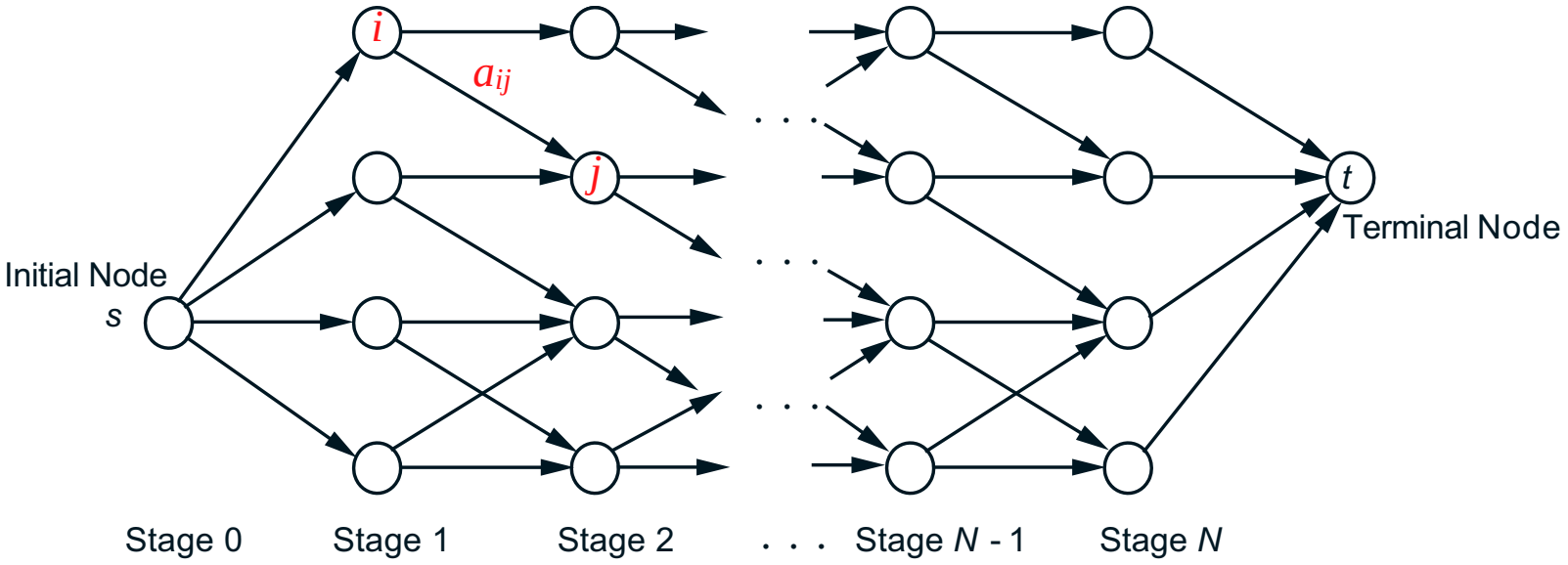
- System divided into stages $k = 0, 1, \cdots, N$
- In stage k, a control $u_k = \mu_k(x_k)$ is taken depending on state x_k
- State $x_k = i \longleftrightarrow$ node i, control $u_k = j \longleftrightarrow$ next node j
- Control sequence $\{u_0, \ldots, u_{N-1}\} \longleftrightarrow$ path from s to t



DP Algorithm

$$J_N(i) = a_{it}, \qquad i \in S^N,$$
$$J_k(i) = \min_{j \in S^{k+1}} \{a_{ij} + J_{k+1}(j)\}, \quad i \in S^k, k = 0, \ldots, N-1$$

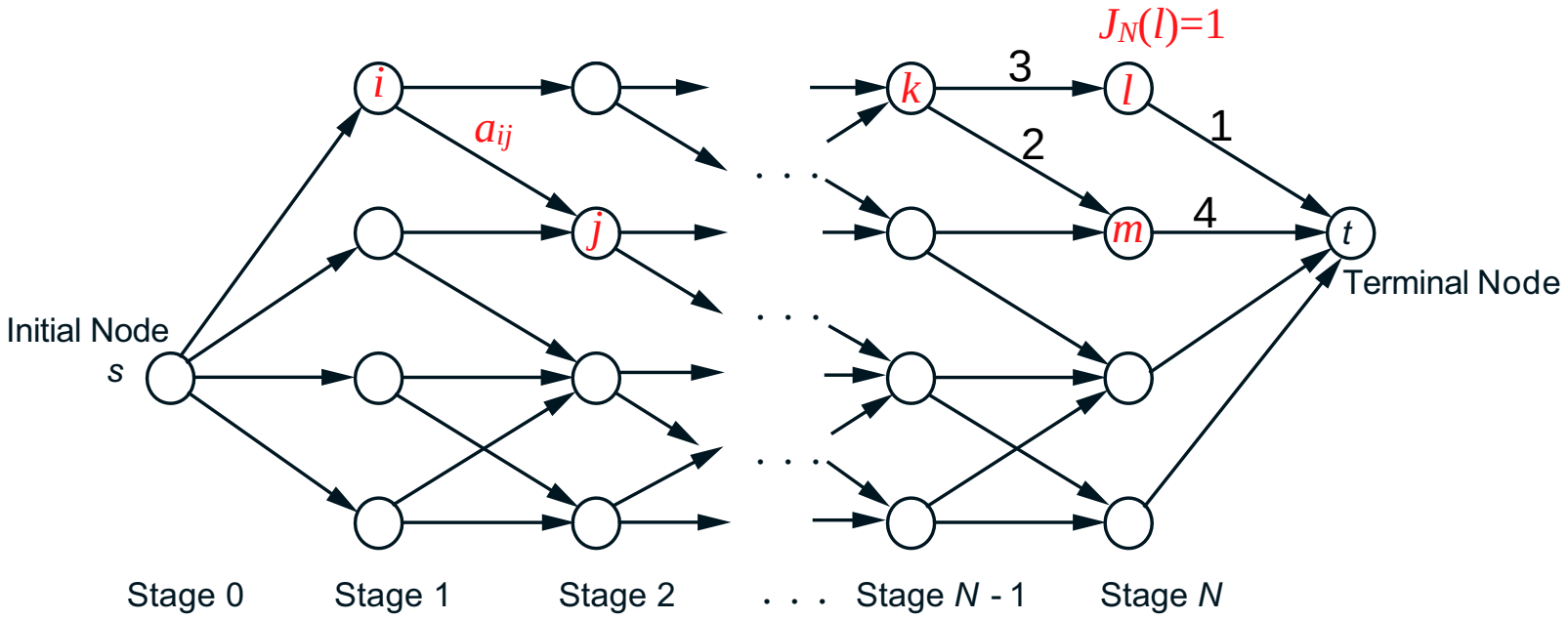
- The minimum cost is equal to $J_0(s)$



DP Algorithm

$$J_N(i) = a_{it}, \qquad i \in S^N,$$
$$J_k(i) = \min_{j \in S^{k+1}} \{a_{ij} + J_{k+1}(j)\}, \quad i \in S^k, k = 0, \ldots, N-1$$

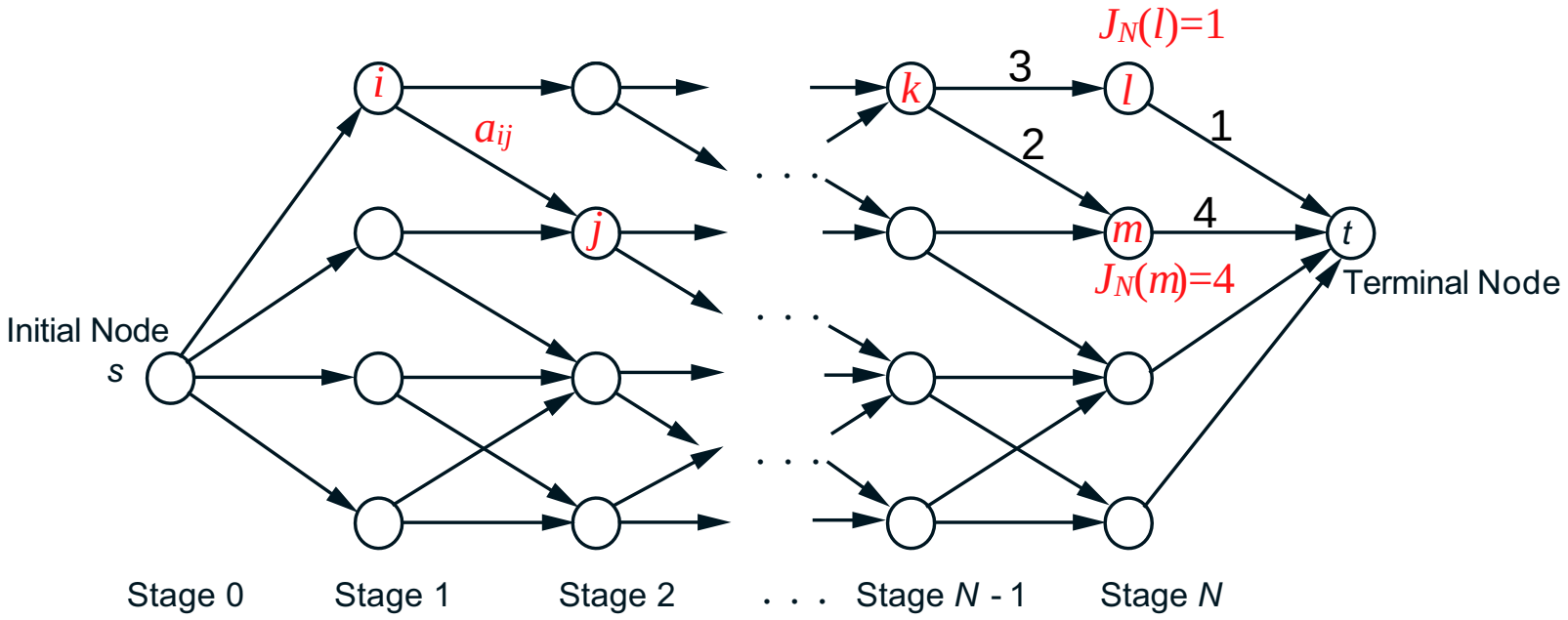
- The minimum cost is equal to $J_0(s)$



DP Algorithm

$$J_N(i) = a_{it}, \qquad i \in S^N,$$
$$J_k(i) = \min_{j \in S^{k+1}} \{a_{ij} + J_{k+1}(j)\}, \quad i \in S^k, k = 0, \ldots, N-1$$

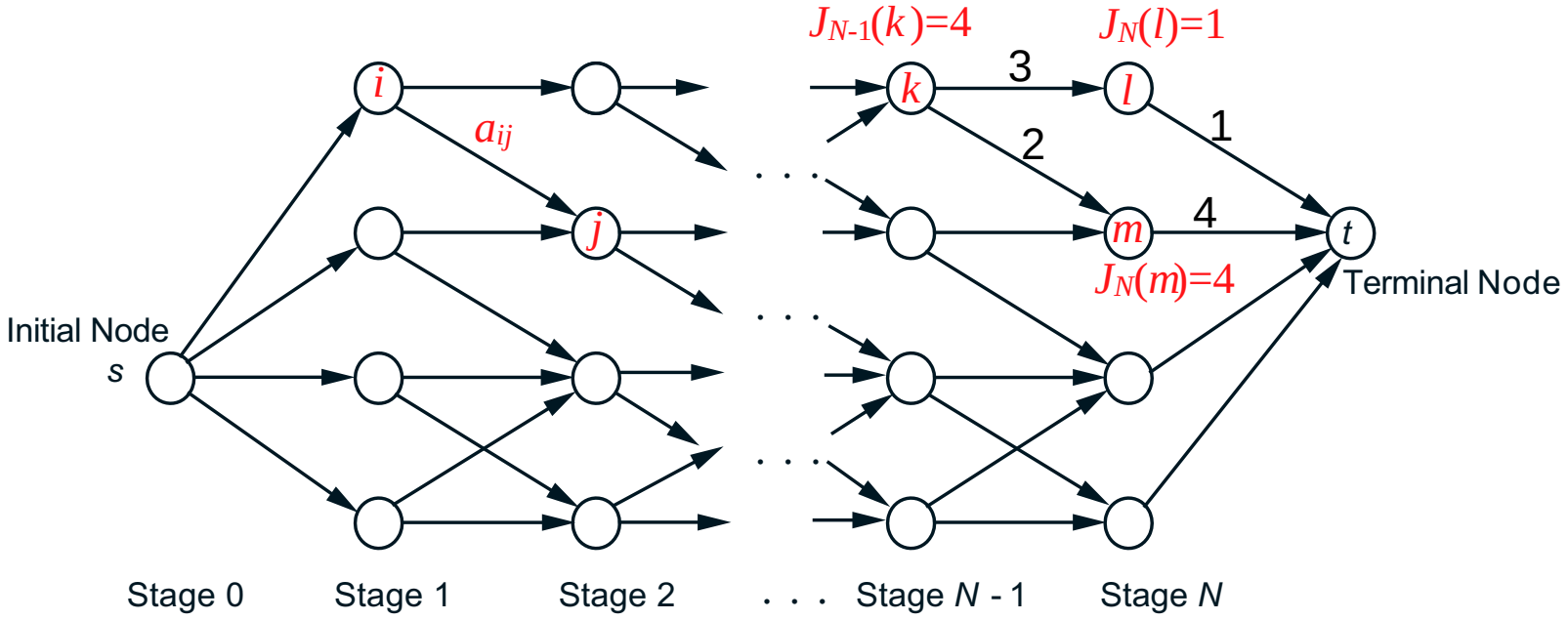
- The minimum cost is equal to $J_0(s)$



DP Algorithm

$$J_N(i) = a_{it}, \qquad i \in S^N,$$
$$J_k(i) = \min_{j \in S^{k+1}} \{a_{ij} + J_{k+1}(j)\}, \quad i \in S^k, k = 0, \ldots, N-1$$

- The minimum cost is equal to $J_0(s)$



Forward DP Algorithm

- Observation: An optimal path $s \to t$ is also an optimal path $t \to s$ in a reverse shortest path problem where the direction of each arc is reversed and its length is left unchanged
- Forward DP algorithm (= backward DP algorithm for the reverse problem):

$$\begin{aligned} \tilde{J}_N(j) &= a_{sj}, & j \in S_1, \\ \tilde{J}_k(j) &= \min_{i \in S_{N-k}} \left\{ a_{ij} + \tilde{J}_{k+1}(i) \right\}, & j \in S_{N-k+1}, k = 1, \ldots, N-1 \end{aligned}$$

- The optimal cost is

$$\tilde{J}_0(t) = \min_{i \in S_N} \left\{ a_{it} + \tilde{J}_1(i) \right\}$$

Application: Hidden Markov Models

- Markov chain with transition probabilities p_{ij}
- State transitions are hidden from view
- For each transition from i to j, we get an (independent) observation z with Prob. $r(z; i, j)$
- Trajectory estimation problem: Given the observation sequence $Z_N = \{z_1, z_2, \ldots, z_N\}$, what is the most likely state transition sequence $\hat{X}_N = \{\hat{x}_0, \hat{x}_1, \ldots, \hat{x}_N\}$

$$\max_{X_N} \quad p(X_N | Z_N)$$

- We have

$$p(X_N | Z_N) = \frac{p(X_N, Z_N)}{p(Z_N)}$$

- Maximizing $p(X_N | Z_N)$ is equivalent with maximizing $p(X_N, Z_N)$

Viterbi Algorithm

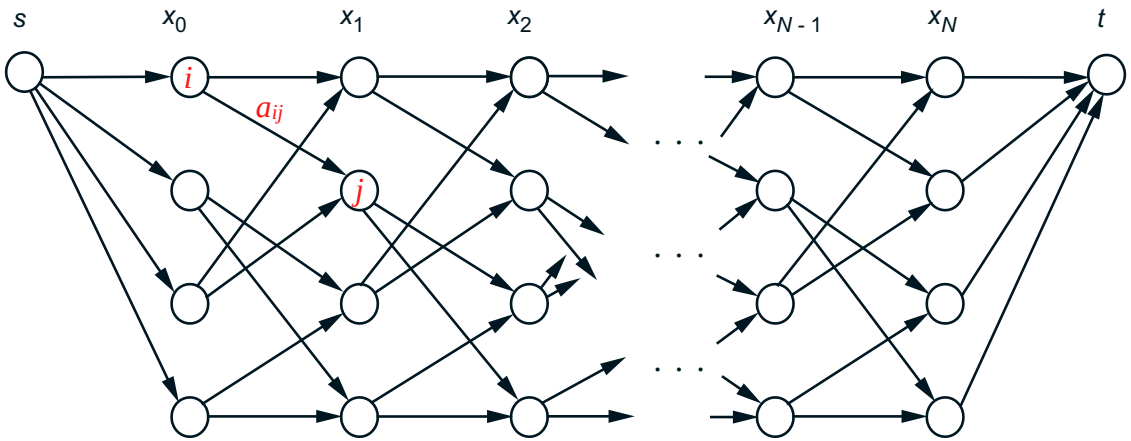
- We have (using the "multiplication rule" for cond. probs)

$$p(X_N, Z_N) = \pi_{x_0} \prod_{k=1}^{N} p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k)$$

- Transform from product to sum by log function

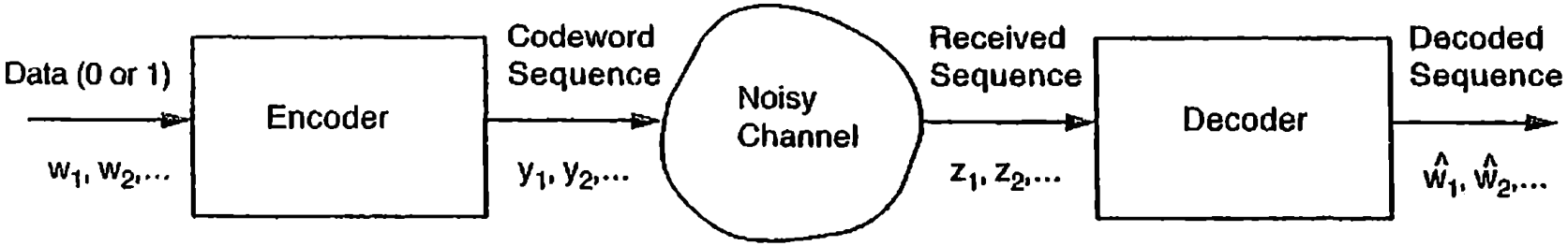
$$\min \quad -\ln(\pi_{x_0}) - \sum_{k=1}^{N} \ln(p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k))$$

- This is a shortest path problem $a_{ij} = -\ln(p_{ij} r(z_k; i, j))$



Example: Convolutional Coding and Decoding

- Noisy communication channel
  - Data sequence $w_k \in \{0, 1\}$
  - Codeword sequence $y_k \in \{0, 1\}^n$
  - Received sequence $z_k \in \{0, 1\}^n$
  - Decoded sequence $\hat{w}_k \in \{0, 1\}$



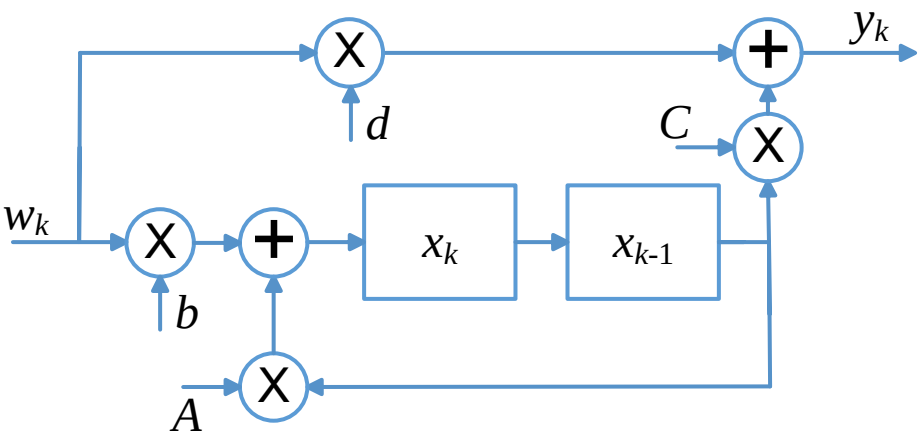
Example: Convolutional Coding and Decoding

- Convolutional Coding

$$y_k = Cx_{k-1} + dw_k, \quad k = 1, 2, \ldots$$

$$x_k = Ax_{k-1} + bw_k, \quad k = 1, 2, \ldots$$

$x_k \in \{0, 1\}^m$ as system state, x_0 is given.

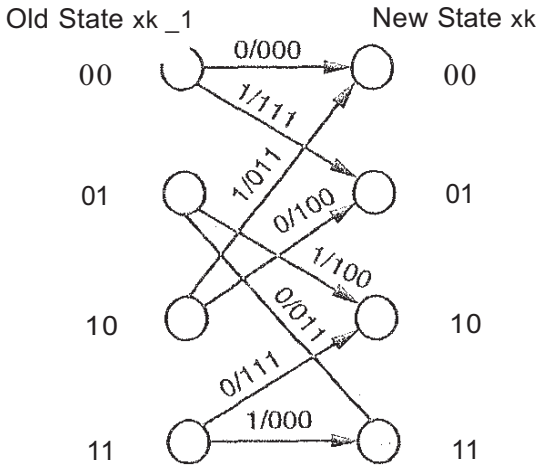


Example: Convolutional Coding and Decoding

- As an example, let $m = 2, n = 3$, and

$$C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, d = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, b = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- State transition



- The pair on each arc is the data/codeword pair w_k/y_k
- Form a trellis diagram.

Example: Convolutional Coding and Decoding

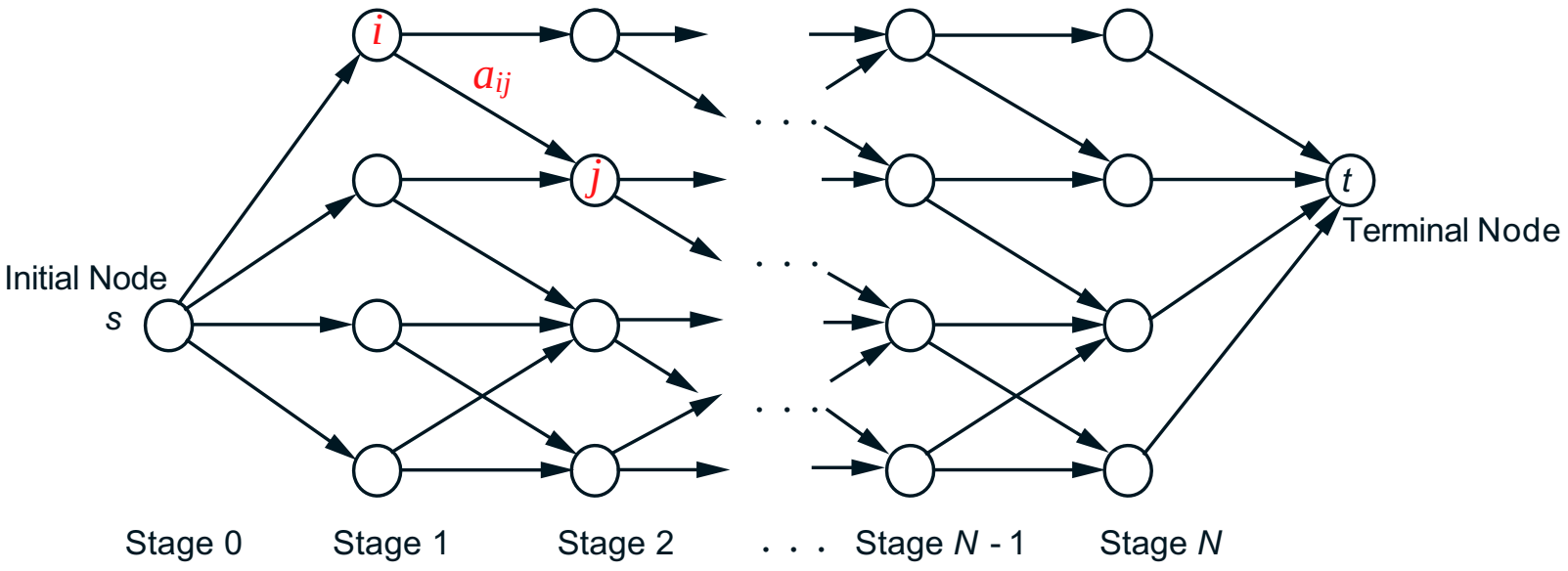
- Maximum likelihood estimation

$$\max_{Y_N} p(Z_N|Y_N) = \max_{y_1,\ldots,y_N} \prod_{k=1}^{N} p(z_k|y_k) \Leftrightarrow \min_{y_1,\ldots,y_N} \sum_{k=1}^{N} -\ln(p(z_k|y_k))$$

- $p(z|y)$ is known (channel characteristics), y_k is associated with state transition
- Shortest path problem (Viterbi Algorithm) in the trellis diagram from s to t, where the length of the arc associated with the codeword y_k is $-\ln(p(z_k|y_k))$
- The corresponding data sequence $\{\hat{w}_1, \ldots, \hat{w}_N\}$ is the decoded data.

General Shortest Path Algorithms

- DP is an exhaustive search algorithm
- There are many non-DP shortest path algorithms
- They may be preferable than DP if they avoid calculating the optimal cost-to-go of every state
- Essential for problems with huge state spaces (AlphaGo)

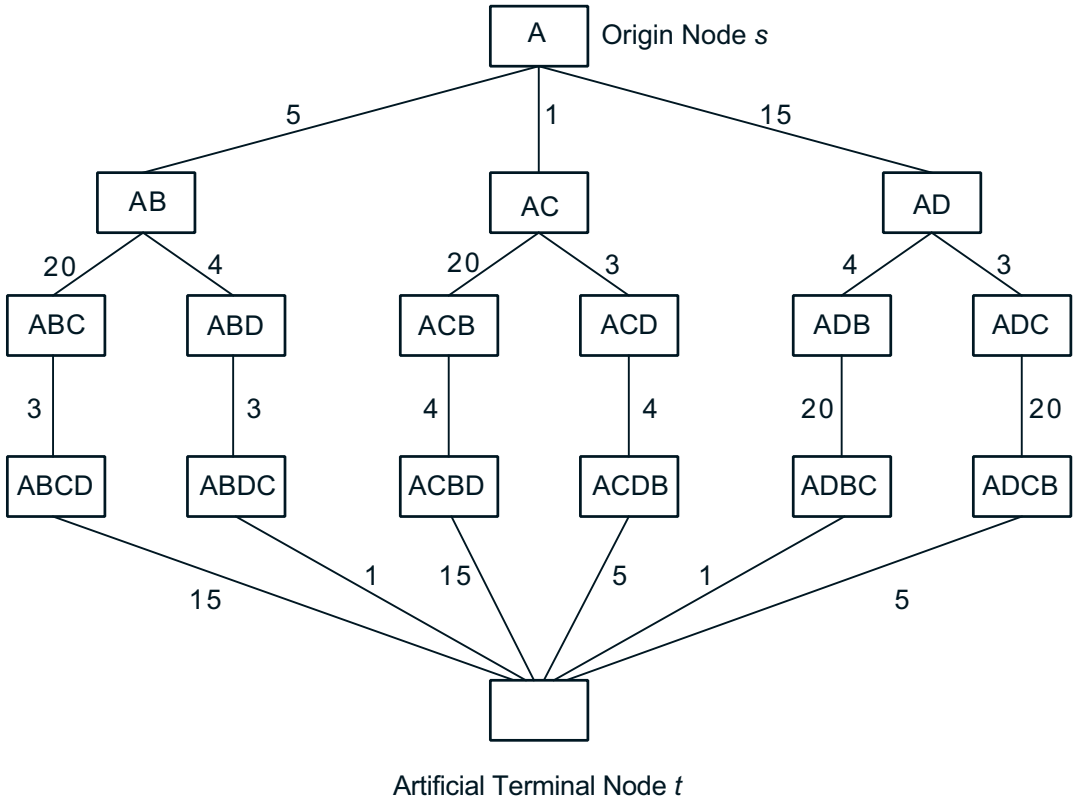


Example: Traveling Salesman Problem

- Travel each of N cities exactly once and return to the origin with a minimum-mileage
- State in stage n: a sequence of n distinct cities
- The number of states grows exponentially with N

	5	1	15
5		20	4
1	20		3
15	4	3	

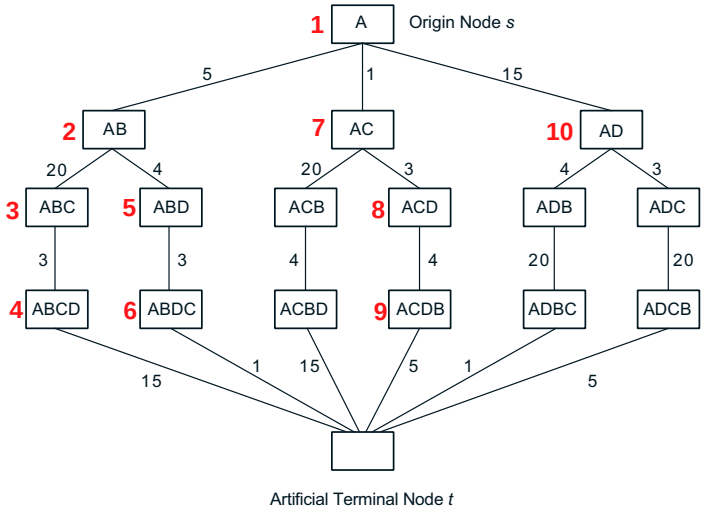
Figure: Distance between the four cities



Label Correcting Methods

- Given: Origin s, destination t, length $a_{ij} \geq 0$.
- Idea is to progressively discover shorter paths from the origin s to every other node i
  - d_i: Length of the shortest path found from s to i (initially $d_s = 0, d_i = \infty$ for $i \neq s$)
  - UPPER: The label d_t of the destination
  - OPEN list: Contains nodes that are currently active in the sense that they are candidates for further examination (initially OPEN$=\{s\}$)
- Step1 (Node Removal): Remove a node i from OPEN and for each child j of i, do step2
- Step2 (Node Insertion Test): If $d_i + a_{ij} < \min\{d_j, \text{UPPER}\}$, set $d_j = d_i + a_{ij}$ and set i to be the parent of j. In addition, if $j \neq t$, place j in OPEN if it is not already in OPEN, while if $j = t$, set UPPER to the new value $d_i + a_{it}$ of d_t
- Step3 (Termination Test): If OPEN is empty, terminate; else go to step1

Label Correcting Methods: Example



Iter. No.	Node Exiting OPEN	OPEN after Iteration	UPPER
0	-	1	∞
1	1	2, 7,10	∞
2	2	3, 5, 7, 10	∞
3	3	4, 5, 7, 10	∞
4	4	5, 7, 10	43
5	5	6, 7, 10	43
6	6	7, 10	13
7	7	8, 10	13
8	8	9, 10	13
9	9	10	13
10	10	Empty	13

Specific Label Correcting Methods

- Based on the order of node removal
- Breadth-first search
  - first-in/first-out
- Depth-first search
  - last-in/first-out
- Best-first search (Dijkstra's method)
  - remove a node with minimum label

$$i = \arg \min_{j \text{ in OPEN}} d_j$$

  - Benefit: a node will enter the OPEN list at most once
  - Overhead: find the node with minimum label in each iteration

Label Correcting Variations

- Non-trivial initialization
  - Initialize a "good" path $P = (s, i_1, \cdots, i_k, t)$
  - Initialize labels as

$$d_i = \begin{cases} \text{Length of portion of path } P \text{ from } s \text{ to } i & \text{if } i \in P \\ \infty & \text{if } i \notin P \end{cases}$$

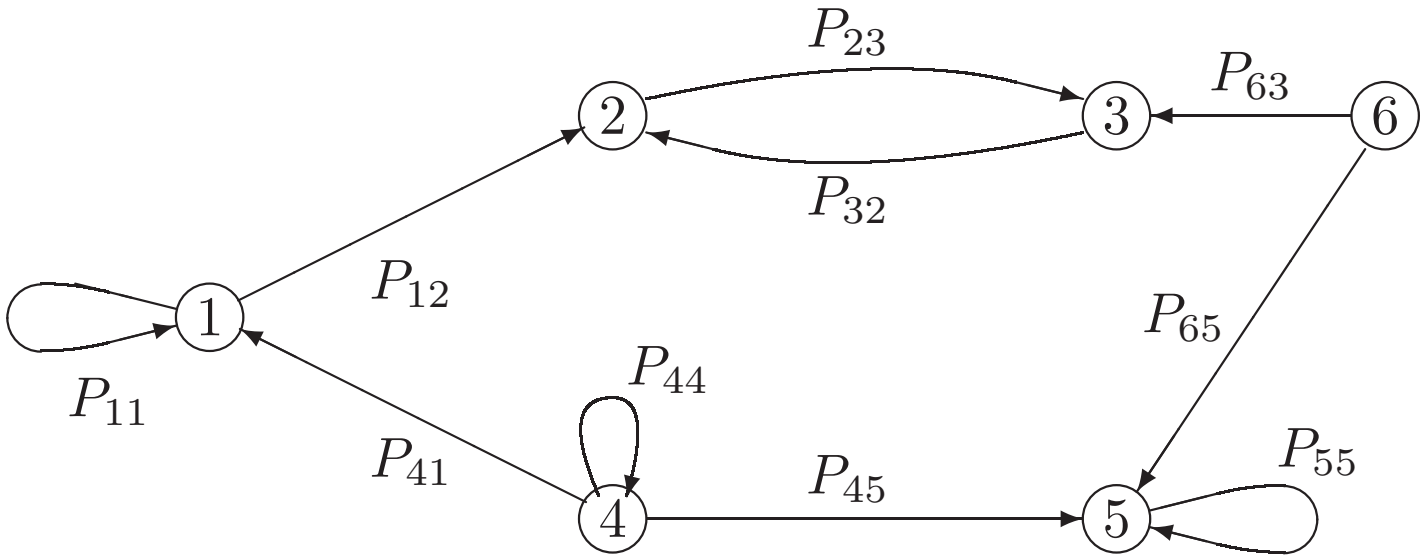
  and UPPER $= \{d_t\}$, OPEN $= \{s, i_1, \cdots, i_k\}$
- A^* Algorithm
  - Place a node j in OPEN only when $d_i + a_{ij} + h_j < \text{UPPER}$, where h_j is a positive underestimate of the shortest distance from j to the destination
- Branch-and-bound
  - Update UPPER if $d_j + m_j < \text{UPPER}$, where m_j is an upper bound of the shortest distance from j to the destination

Outline

1. Shortest Path Problem
2. Markov Decision Process
3. Remarks

Review of Markov Chain

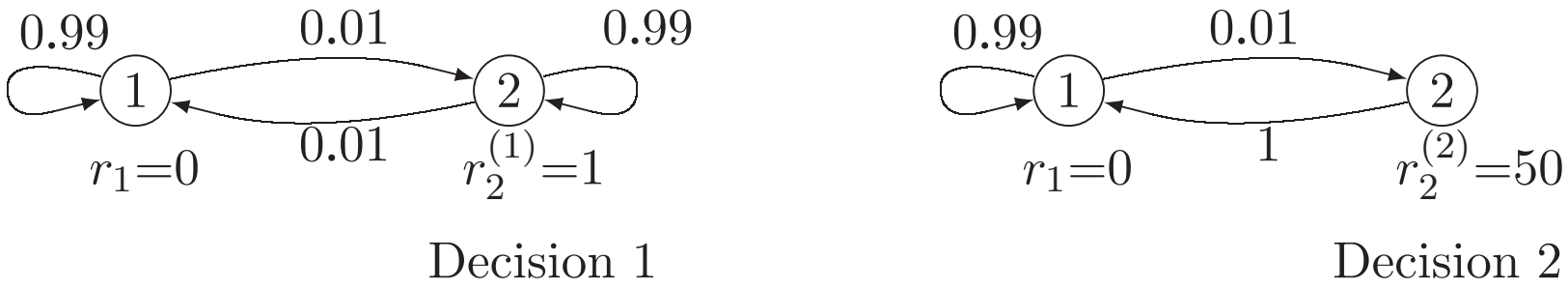
- Example of Markov Chain:



- In many problems, the state transition can be controlled
- Control decision in each state incurs a cost/reward

Markov Decision Process

- Markov decision process (MDP): a "controlled" Markov chain



- Objective: determine a set of state-dependent decisions to minimize total cost/maximize total reward
- Problem formulation: long-term optimization problem
- Tradeoff between instant gratification and long-term gratification

Markov Decision Process (Finite Horizon)

- Discrete-time system with stage k

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad k = 0, 1, \ldots, N-1$$

- x_k: State of the system (also termed as s_k)
- u_k: Action to be selected (also termed as a_k)
- f_k: State transition function with random parameter w_k, usually described implicity by controlled transition prob. $p_{ij}(u)$
- g_k: Cost function that is additive over time

$$g_k(x_k, u_k, w_k), \quad k = 0, 1, \ldots, N-1 \text{ and } g_N(x_N)$$

Problem Formulation

- Policy: $\pi = \{\mu_0, \ldots, \mu_{N-1}\}$ (maybe random), such that

$$u_k = \mu_k(x_k) \in U_k(x_k)$$

- Expected cost of π starting at x_0 is

$$J_\pi(x_0) = E\left\{\sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k) + g_N(x_N)\right\}$$

- Objective: find optimal policy π^* to achieve minimum cost J^*

$$J^*(x_0) = \min_\pi \; J_\pi(x_0) = J_{\pi^*}(x_0)$$

- Remark: Shortest path problem is a deterministic MDP

Markovian Assumptions

- u_k depends at most on x_k and not on prior x or u

$$u_k \in U_k(x_k)$$

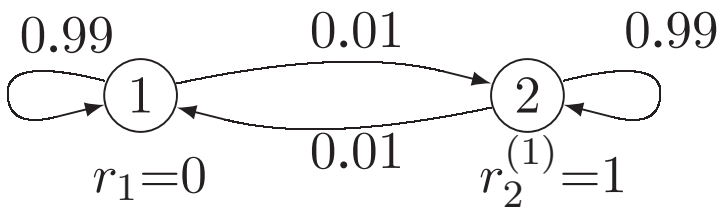
- Probability distribution of w_k does not depend on past values $w_{k-1}, \ldots, w_0$ but may depend on x_k and u_k

$$P_k(w_k | x_k, u_k)$$

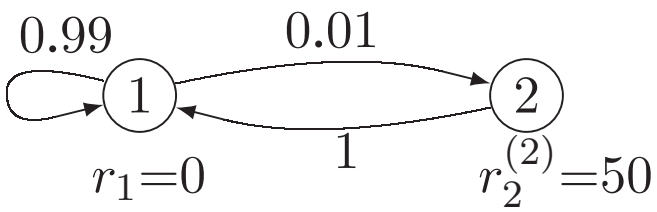
- State transition depends only on the current state, control and randomness

$$x_{k+1} = f_k\left(x_k, u_k, w_k\right)$$

Example 1: Controlled Markov Chain



Decision 1



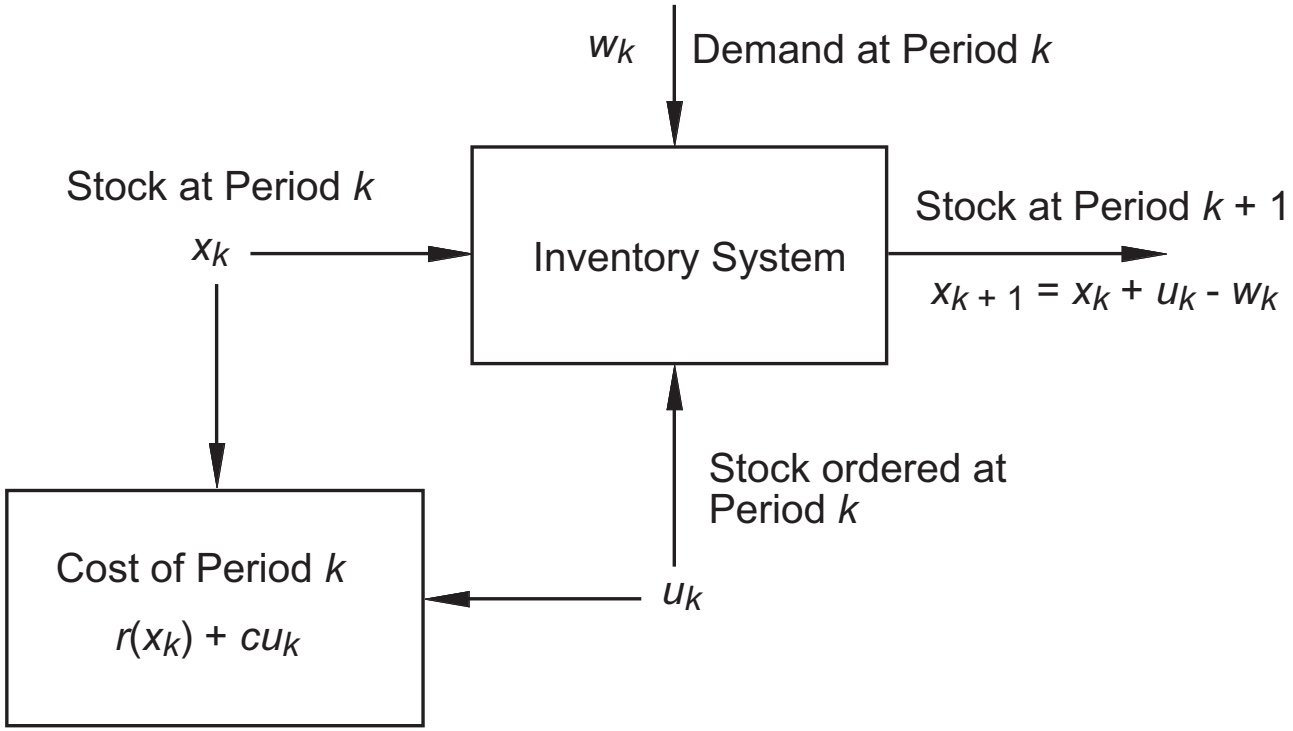
Decision 2

- State: $x_k \in \{1, 2\}$
- Action: $U(1) = \{1\}, U(2) = \{1, 2\}$
- State transition: $p_{ij}(u)$
- Cost function: $r_1 = 0, r_2^{(1)} = 1, r_2^{(2)} = 50$
- Objective:

$$\min \ E\left(\sum_{k=0}^{N} r_{x_k}^{(u_k)}\right)$$

Example 2: Inventory Control

- Ordering a quantity of a certain item in each period to meet a stochastic demand, while minimizing the expected cost



Example 2: Inventory Control

- State: stock x_k available at the beginning of period k
- Action: stock u_k ordered at the beginning of period k
- State transition: $x_{k+1} = x_k + u_k - w_k$, where w_k is the demand during period k
- Cost function: stock cost $r(x_k)$ and purchasing cost cu_k
- Objective:

$$\min_{\mu_0, \cdots \mu_{N-1}} E\left\{\sum_{k=0}^{N-1} (c\mu_k(x_k) + r(x_k)) + R(x_N)\right\}$$

Example 3: Power Control

- Transmit M packets in N stages ($N \geq M$), at most one packet can be transmitted in each stage
- Finite state Markov channel $h_k \in \{0, 1, \cdots, L\}$
- Cost (transmit power) to transmit a packet: $\infty = p(0) > p(1) > p(2) > \cdots > p(L)$
- Cost for remaining packet(s) at the end of N-th slot: K

Example 3: Power Control

- State: $x_k = (m_k, h_k)$, m_k - transmitted packets, h_k - channel state
- Action u_k: 1 (transmit) or 0 (not transmit)
- State transition: $m_{k+1} = m_k + u_k, h_{k+1} \sim \Pr(h_{k+1}|h_k)$
- Cost function:

$$g_k(x_k, u_k) = u_k p(h_k), \quad k < N$$

$$g_N(x_N) = K\mathrm{I}_{m_N < M}$$

Example 4: Scheduling Problem

- Find optimal sequence of operations A,B,C,D
- A must precede B, and C must precede D
- Start up cost S_A, S_C
- Transition cost C_{mn} from operation m to operation n

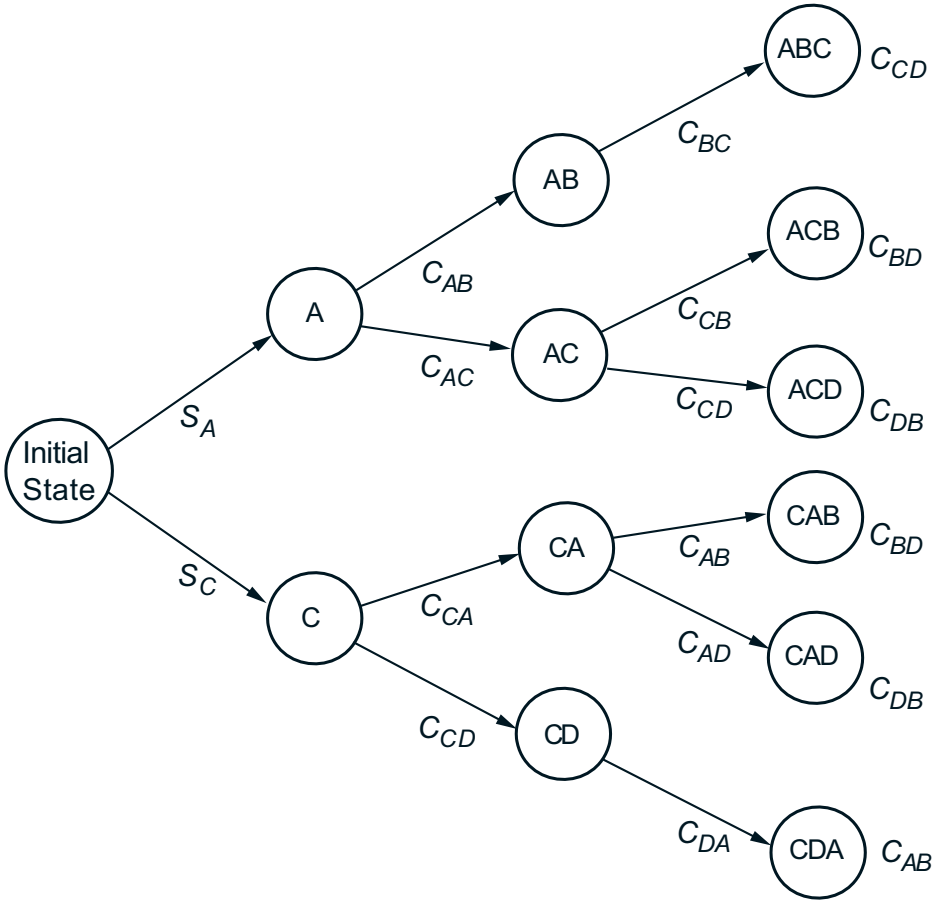
Example 4: Scheduling Problem

- State $x_k = (u_1, \cdots, u_{k-1})$: series of operations up to stage $k \in \{1, \cdots, 4\}$, $x_1 = \emptyset$
- Action u_k: operation in stage k
- State transition: $x_{k+1} = (x_k, u_k)$ (deterministic)
- Cost function:

$$g_1(u_1) \in \{S_A, S_C\}$$

$$g_k(x_k, u_k) = C_{u_{k-1}u_k}, \quad k = 2, 3, 4$$

Example 4: Scheduling Problem (Deterministic)



Principle of optimality

- Let $\pi^* = \{\mu_0^*, \mu_1^*, \ldots, \mu_{N-1}^*\}$ be optimal policy
- Consider the tail subproblem whereby we are at x_i at time i and wish to minimize the cost-to-go from time i to time N

$$E\left\{\sum_{k=i}^{N-1} g_k(x_k, \mu_k(x_k), w_k) + g_N(x_N)\right\}$$

  and the "tail policy" $\{\mu_i^*, \mu_{i+1}^*, \ldots, \mu_{N-1}^*\}$
- *Principle of optimality* (Bellman): The tail policy is optimal for the tail subproblem (optimization of the future does not depend on what we did in the past)
- Dynamic Programming (DP) algorithm is based on this principle

DP Algorithm

- Start with
$$J_N(x_N) = g_N(x_N),$$
and go backwards for $k = N-1, N-2, \cdots, 0$
$$J_k(x_k) = \min_{u_k} E_{w_k} \left\{ g_k(x_k, u_k, w_k) + J_{k+1}\left(f_k(x_k, u_k, w_k)\right) \right\},$$
where $\mu_k^*(x_k)$ minimizes the right side for each x_k
- $J_0(x_0)$ is equal to the optimal cost $J^*(x_0)$, and the optimal policy is
$$\pi^* = \{\mu_0^*, \ldots, \mu_{N-1}^*\}$$
- Question: Computational Complexity? Storage Requirement?

Deterministic v.s. Stochastic

- Deterministic DP

$$
\begin{aligned}
J_N(i) &= a_{it}, \\
J_k(i) &= \min_{j \in S^{k+1}} \left\{ a_{ij} + J_{k+1}(j) \right\},
\end{aligned}
$$

for $k = N-1, N-2, \cdots, 0$

- Stochastic DP

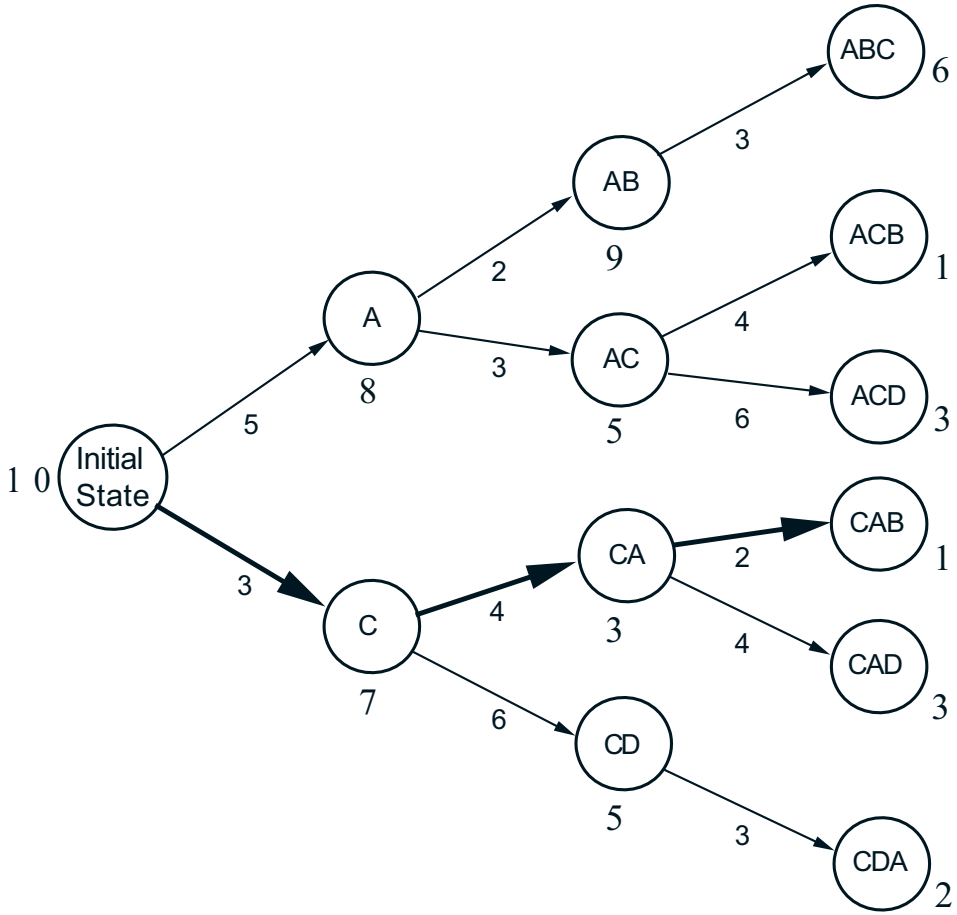
$$
\begin{aligned}
J_N(x_N) &= g_N(x_N), \\
J_k(x_k) &= \min_{u_k} E_{w_k} \left\{ g_k(x_k, u_k, w_k) + J_{k+1}\left(f_k(x_k, u_k, w_k)\right) \right\},
\end{aligned}
$$

for $k = N-1, N-2, \cdots, 0$

- Deterministic DP is a special case of Stochastic DP

Example 4: Solving Scheduling Problem

from \ to	A	B	C	D
A	-	2	3	4
B	-	-	3	1
C	4	4	-	6
D	3	3	-	-



- At each stage, we record the optimal cost-to-go and the optimal decision for each state

State Augmentation

- When Markovian assumptions are violated (e.g., disturbances are correlated, etc), reformulate/augment the state
- DP algorithm still applies, but the problem gets BIGGER
- Example: Historical dependent

$$x_{k+1} = f_k(x_k, x_{k-1}, u_k, w_k)$$

- Introduce additional state variable $y_k = x_{k-1}$. New system takes the form

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} f_k(x_k, y_k, u_k, w_k) \\ x_k \end{pmatrix}$$

- View $\tilde{x}_k = (x_k, y_k)$ as the new state, the problem is an MDP with larger state space

Outline

1. Shortest Path Problem
2. Markov Decision Process
3. Remarks

Remarks

- Very powerful tool for sequential decision making
  - Insightful and well-structured solutions (Week 13)
  - Generalized problem for infinite horizon and iterative solution (Week 14)
- Model-based (should know state transition, cost function, etc.)
- The curse of dimensionality (e.g., AlphaGo, state augment, etc.)
  - Approximate DP (ADP) or Reinforcement learning (Week 15)

Homework

Consider the system

$$x_{k+1} = x_k + u_k + w_k, \qquad k = 0, 1, 2, 3,$$

with initial state $x_0 = 5$, and the cost function

$$\sum_{k=0}^{3} (x_k^2 + u_k^2).$$

Apply the DP algorithm for the following three cases:

(a) The control constraint set $U_k(x_k)$ is $\{u \mid 0 \leq x_k + u \leq 5,\ u : \text{integer}\}$ for all x_k and k, and the disturbance w_k is equal to zero for all k.

(b) The control constraint is as in part (a) and the disturbance w_k takes the values -1 and 1 with equal probability $1/2$ for all x_k and u_k, except if $x_k + u_k$ is equal to 0 or 5, in which case $w_k = 0$ with probability 1.